

フロントエンド変更解説 PDF

対象日: 2026-06-09 / 対象: React + Vite フロントエンドの未コミット差分

この資料は、現在の差分をファイルごとに読み解くための学習用メモです。以降、Codex はアプリのソースコードを直接編集せず、レビュー・解説・ヒント出しに回る前提でまとめています。

ファイル	変更の役割
frontend/src/App.tsx	アプリ全体の器に整理し、HomeScreen を表示する入口にした
frontend/src/App.css	ページ全体の背景と最小高さを担当する新規 CSS
frontend/src/screens/HomeScreen.tsx	ホーム画面にヘッダー、検索、カテゴリ、おすすめ一覧を追加
frontend/src/screens/HomeScreen.css	ホーム画面の余白、グリッド、レスポンスを担当
frontend/src/components/ItemCard.tsx	商品カードの情報表示を増やし、価格やカテゴリをデータから表示
frontend/src/components/ItemCard.css	カードの見た目、状態ラベル、画像、ホバー、文字省略を整備
frontend/src/features/items/itemTypes.ts	Item 型に price を追加
frontend/src/features/items/itemData.ts	デモ商品の price を追加
frontend/src/styles/globals.css	画像の基本スタイルを追加
frontend/tsconfig.json	TypeScript がプロジェクトを検査できるように設定を追加
frontend/dist-file-preview/	ファイルプレビュー用に生成したビルド成果物。学習用ソースではない

frontend/src/App.tsx

役割: アプリ全体の入口です。画面コンポーネントをどこに表示するかを決めます。

主な変更

- ・ App.css を import して、アプリ全体に CSS を当てられるようにした。
- ・ main に app-shell クラスを付け、ページ全体の背景や高さを CSS 側に任せた。
- ・ 仮の h1、h2、Current screen 表示を削り、HomeScreen だけを表示する構造にした。
- ・ data-current-screen に currentScreen を残し、将来デバッグや CSS 制御に使える余地を残した。

学びポイント

- ・ App は細かい UI を詰め込む場所ではなく、ページやルーティングを接続する場所として使うと見通しが良くなる。
- ・ UI の見た目は className を通して CSS に逃がすと、JSX は構造を読むことに集中できる。

自分で書くときの注目点

自分で書くなれば、まず「このファイルは画面を組み立てる係」と意識して、細かいカード HTML などを入れすぎないのがコツです。

代表スニペット

```
return (  
  <main className="app-shell" data-current-screen={currentScreen}>  
    <HomeScreen />  
  </main>  
);
```

frontend/src/App.css

役割: ページ全体の背景と最低高さを担当する新規ファイルです。

主な変更

- ・ app-shell に min-height: 100vh を指定し、画面の高さが足りない場合でも背景が途切れないようにした。
- ・ linear-gradient と #f6f7fb を重ね、上部だけ少し青みのある背景にした。

学びポイント

- ・ 画面全体の背景は App 直下の shell に置くと、個別コンポーネントの責務が軽くなる。
- ・ CSS の background は複数レイヤーをカンマで重ねられる。先に書いたものが手前に描画される。

自分で書くときの注目点

背景の調整は「色を足す」より先に、どの領域に効かせたいかを決めると崩れにくいです。

代表スニペット

```
.app-shell {  
  min-height: 100vh;  
  background: linear-gradient(...), #f6f7fb;  
}
```

frontend/src/screens/HomeScreen.tsx

役割: ホーム画面の構造を担当します。ここでは商品一覧だけでなく、ヘッダー、検索、カテゴリ、一覧見出しまで持たせています。

主な変更

- ・ categories 配列を追加し、カテゴリボタンを map で描画する形にした。
- ・ header を追加し、サービス名、説明文、出品ボタンを配置した。
- ・ 検索 UI を role="search" 付きで追加した。
- ・ カテゴリ nav を aria-label 付きで追加した。
- ・ おすすめ見出しと件数表示を追加した。
- ・ demoItems.map の位置を grid の中に移し、カード一覧として扱いやすくした。

学びポイント

- ・ map は「配列データを UI に変換する」React の基本パターン。カテゴリも商品も同じ考え方で描画できる。
- ・ 画面単位のコポーネントは、部品を並べる責務を持つ。ItemCard の中身まではここに書かない。
- ・ role や aria-label は見た目に出ないが、アクセシビリティ上の意味を補う。

自分で書くときの注目点

次に自分で書くなら、カテゴリをクリックしたときに activeCategory を state で持ち、表示商品を filter する練習がよいです。

代表スニペット

```
{demoItems.map((item) => (  
<ItemCard key={item.id} item={item} />  
))}
```

frontend/src/screens/HomeScreen.css

役割: ホーム画面全体のレイアウト、余白、検索欄、カテゴリ、グリッド、レスポンスを担当します。

主な変更

- ・ home-screen の最大幅を 420px から 960px に広げ、PC でも一覧画面として使える幅にした。
- ・ ヘッダー周辺の typography と余白を追加した。
- ・ 検索欄を横並び flex で作り、入力欄が親幅に収まるよう min-width: 0 を入れた。
- ・ カテゴリボタンを横スクロール可能にした。スマホでカテゴリが増えても崩れにくい。
- ・ 商品一覧を CSS Grid にし、通常は 2 列、760px 以上では 3 列、520px 以下では 1 列にした。

学びポイント

・ 画面レイアウトには flex と grid を使い分ける。横並びの小さい要素は flex、カード一覧のような面は grid が向く。

- ・ `minmax(0, 1fr)` はグリッド内の要素がはみ出す事故を減らすための実務的な書き方。
- ・ メディアクエリは「見た目が苦しくなる幅」に合わせて増やすと自然。

自分で書くときの注目点

自分で調整するときは、まず `.home-screen__grid` の列数と `gap` を触ると、画面の印象が大きく変わります。

代表スニペット

```
.home-screen__grid {  
  display: grid;  
  grid-template-columns: repeat(2, minmax(0, 1fr));  
  gap: 16px;  
}
```

frontend/src/components/ItemCard.tsx

役割: 1 件の商品データをカード UI に変換する部品です。

主な変更

- ・画像の alt を空文字から item.title に変更し、画像の意味が伝わるようにした。
- ・いいねのハートを aria-hidden にして、読み上げ対象を数値中心にした。
- ・状態ラベルとカテゴリを item-card__meta にまとめた。
- ・価格を固定値の \25,000 から item.price.toLocaleString() に変更した。
- ・お気に入りボタンに aria-label を追加した。
- ・ボタン表示を ☆ に変更し、ボタンの意味は aria-label で補った。

学びポイント

- ・ props で受け取った item を使えば、カードはどの商品にも再利用できる。
- ・ 固定テキストをデータ参照に変えるほど、コンポーネントは実アプリに近づく。
- ・ toLocaleString() は数値を 25,000 のように読みやすく整形する。

自分で書くときの注目点

あなたがこのファイルを直すなら、まず「表示したいデータは Item 型にあるか」を確認してから JSX に置くと迷いにくいです。

代表スニペット

```
¥{item.price.toLocaleString()}
```

frontend/src/components/ItemCard.css

役割: 商品カードの見た目を細かく担当します。カードは再利用されるので、この CSS の品質が一覧全体の見やすさに直結します。

主な変更

- ・ カードに border と薄い shadow を追加し、背景から少し浮いて見えるようにした。
- ・ hover 時に shadow と transform を変え、操作できそうな感触を加えた。
- ・ 画像領域の aspect-ratio を調整し、カード内の写真比率を安定させた。
- ・ item-card__meta を追加し、状態ラベルとカテゴリを左右に整理した。
- ・ タイトルを 2 行で省略する line-clamp を入れ、長いタイトルでカード高さが大きく崩れないようにした。
- ・ wanted と owner は 1 行省略にして、カードの情報密度を安定させた。
- ・ 価格を少し強めにし、お気に入りボタンに hover 表現を追加した。

学びポイント

- ・ カード UI は「画像」「メタ情報」「本文」「アクション」のブロックに分けると CSS が書きやすい。
- ・ overflow, text-overflow, white-space は長い文字列対策の基本セット。

- ・hover は飾りではなく、押せる・選べるという手がかりとして使う。

自分で書くときの注目点

ここを自分で書くときは、一気に全部ではなく、画像領域、本文、下部アクションの順で確認すると理解が深まります。

代表スニペット

```
.item-card__title {
display: -webkit-box;
-webkit-line-clamp: 2;
overflow: hidden;
}
```

frontend/src/features/items/itemTypes.ts

役割: 商品データの型定義です。React 側が期待するデータの形をここで決めます。

主な変更

- ・Item 型に price: number を追加した。

学びポイント

- ・TypeScript では UI で使うデータを型に追加しておく、データ不足を早く検出できる。
- ・price は計算や整形をしやすいように string ではなく number にしている。

自分で書くときの注目点

表示を増やしたいときは、まず型にフィールドを追加し、次にデータ、最後にコンポーネントの順に進むと安全です。

代表スニペット

```
price: number;
```

frontend/src/features/items/itemData.ts

役割: 画面確認用のデモ商品データです。バックエンド接続前はこのファイルが仮 API のような役割になります。

主な変更

- ・1 件目の商品に price: 25000 を追加した。
- ・2 件目の商品に price: 7800 を追加した。

学びポイント

- ・型に price を追加したら、Item[] の各要素にも price が必要になる。これが型とデータのつながり。
- ・デモデータは UI の確認精度を上げるために、実際に起こりそうな値を入れるとよい。

自分で書くときの注目点

今後は status が completed の商品や、長い title の商品も追加すると、カードが崩れないか練習できます。

代表スニペット

```
price: 25000,
```

frontend/src/styles/globals.css

役割: アプリ全体に効く基本 CSS です。個別コンポーネントではなく全体の土台を作ります。

主な変更

- ・ `img { max-width: 100%; }` を追加した。

学びポイント

- ・ 画像が親要素より大きく表示されてレイアウトを壊すのを防ぐための基本設定。
- ・ `globals.css` は強力なので、個別部品に閉じない本当に共通のルールだけを書くのが大切。

自分で書くときの注目点

グローバル CSS に書く前に、「すべての画面で同じルールにしたいか」を自問するとよいです。

代表スニペット

```
img {  
  max-width: 100%;  
}
```

frontend/tsconfig.json

役割: TypeScript の設定ファイルです。ビルド時に型チェックするためのプロジェクト設定を定義します。

主な変更

- ・ Vite + React 向けの `compilerOptions` を追加した。
- ・ `strict: true` で型チェックを厳しめにした。
- ・ `jsx: react-jsx` を指定し、React の JSX を変換できるようにした。
- ・ `include: ["src"]` で `src` 配下を型チェック対象にした。

学びポイント

- ・ `tsc --noEmit` は `tsconfig.json` を読んで、JS を出力せずに型だけ確認する。
- ・ 設定ファイルがないと `tsc` はヘルプを表示して止まることがある。今回のビルド失敗はこのパターン。

自分で書くときの注目点

今は細かい設定を全部暗記しなくて大丈夫です。まず「TypeScript のルールブック」と捉えるのが第一歩です。

代表スニペット

```
"strict": true,  
"jsx": "react-jsx"
```

frontend/dist-file-preview/

役割: ローカル確認用に生成されたビルド成果物です。手書きで学ぶ対象のソースコードではありません。

主な変更

- ・ index.html、assets 配下の JS/CSS、images/demo 配下の画像が生成された。
- ・ npm exec vite -- build --base ./ --outDir dist-file-preview によって作られたプレビュー用ファイル。

学びポイント

- ・ dist 系フォルダは通常、開発者が直接編集する場所ではなく、ビルドツールが作る出力先。
- ・ Git 管理に含めるかはチーム方針次第。多くの Vite プロジェクトでは dist は .gitignore に入れる。

自分で書くときの注目点

自分で触るなら src 配下を編集します。dist-file-preview は確認用の結果として扱えば十分です。

代表スニペット

```
frontend/dist-file-preview/assets/index-*.js  
frontend/dist-file-preview/assets/index-*.css
```

今日の理解チェック

- App.tsx に細かいカード UI を置かず、HomeScreen に画面の中身を任せる理由を説明できる。
- demoItems.map が、配列からカード一覧を作っていることを説明できる。
- Item 型に price を追加すると itemData.ts にも price が必要になる理由を説明できる。
- CSS Grid と Flexbox の使い分けを、今回の画面で説明できる。
- dist-file-preview は編集対象ではなくビルド成果物だと判断できる。

次に自分で進めるなら

1. HomeScreen.tsx に activeCategory の state を追加する。
2. カテゴリボタンをクリックしたら activeCategory を変える。
3. demoItems を filter して、選択カテゴリの商品だけ表示する。
4. ItemCard.css の余白や影を自分の好みに調整する。